```
CREATE DATABASE IF NOT EXISTS Credito;
USE Credito;

--
================================================================================
-- 1. CREACIÓN DE TABLAS (DDL)
--
================================================================================

CREATE TABLE UsuarioAcceso (
    idUsuarioAcceso INT NOT NULL AUTO_INCREMENT,
    clave VARCHAR(200),
    salt VARCHAR(100),
    nombreUsuario VARCHAR(40),
    numeroIntentos INT NOT NULL DEFAULT 0,
    ultimoIntento DATE,
    PRIMARY KEY (idUsuarioAcceso)
);

CREATE TABLE UsuarioCliente (
    idCliente INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(40),
    ingreso DECIMAL(10,3),
    gastos DECIMAL(10,3),
    historial VARCHAR(100),
    estado TINYINT NOT NULL DEFAULT 3,
    PRIMARY KEY (idCliente)
);

CREATE TABLE Empresa (
    idEmpresa INT NOT NULL AUTO_INCREMENT,
    rubro VARCHAR(50),
    director VARCHAR(40),
    idCliente INT NOT NULL,
    FOREIGN KEY (idCliente) REFERENCES UsuarioCliente(idCliente),
    PRIMARY KEY (idEmpresa)
);

CREATE TABLE Persona (
    idPersona INT NOT NULL AUTO_INCREMENT,
    idCliente INT NOT NULL,
    FOREIGN KEY (idCliente) REFERENCES UsuarioCliente(idCliente),
    PRIMARY KEY (idPersona)
);

CREATE TABLE infoLog (
```

```
    idLog INT NOT NULL AUTO_INCREMENT,
    numeroError INT,
    error VARCHAR(300),
    idUsuarioAcceso INT,
    FOREIGN KEY (idUsuarioAcceso) REFERENCES UsuarioAcceso(idUsuarioAcceso),
    PRIMARY KEY (idLog)
);

CREATE TABLE Solicitud (
    idSolicitud INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    ingresoAcutla INT NOT NULL,
    gastoAnual DECIMAL(10,2) NOT NULL,
    estadoSolicitud VARCHAR(20) NOT NULL,
    fechaSolicitud DATE NOT NULL,
    idCliente INT NOT NULL,
    FOREIGN KEY (idCliente) REFERENCES UsuarioCliente(idCliente)
);

CREATE TABLE estado (
    idEstado INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    estadoCredito VARCHAR(20) NOT NULL,
    idCliente INT NOT NULL,
    FOREIGN KEY (idCliente) REFERENCES UsuarioCliente(idCliente)
);



--
================================================================================
-- 2. STORED PROCEDURES (DML)
--
================================================================================

DELIMITER //

-- CARGAR LOG
CREATE PROCEDURE `Cargar_Log`(
    IN `Numero_Error_IN` INT,
    IN `Mensaje_IN` VARCHAR(500),
    IN `Proceso_IN` VARCHAR(50),
    IN `ID_Usuario_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
```

```
  INSERT INTO infoLog (numeroError, error, idUsuarioAcceso)
  SELECT Numero_Error_IN, LEFT(CONCAT(Proceso_IN,' ',Mensaje_IN),300), (SELECT
idUsuarioAcceso FROM UsuarioAcceso WHERE idUsuarioAcceso = ID_Usuario_IN);
END //


-- INICIALIZADOR
CREATE PROCEDURE `Inicializador`()
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
   DECLARE validador TINYINT DEFAULT 0;

   DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
    ROLLBACK;
    BEGIN
       DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
      CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Inicializador()',NULL);
    END;
    IF validador THEN
       SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
    END IF;
    RESIGNAL;
  END;

  IF NOT EXISTS (SELECT 1 FROM UsuarioAcceso WHERE idUsuarioAcceso = 1) THEN
     START TRANSACTION;
       INSERT INTO UsuarioAcceso (idUsuarioAcceso, nombreUsuario, clave, salt,
numeroIntentos, ultimoIntento)
      VALUES (1, 'admin', 'CAMBIAR_CLAVE', 'CAMBIAR_SALT', 0, CURRENT_DATE());
       CALL Cargar_Log(0,'Se creó el usuario administrador con ID = 1','[SQL]
Inicializador()',1);
    COMMIT;
  ELSEIF NOT EXISTS (SELECT 1 FROM UsuarioAcceso WHERE idUsuarioAcceso = 1
AND nombreUsuario = 'admin') THEN
     SELECT 'El ID 1 está ocupado por un usuario que no es el administrador.' AS
'ADVERTENCIA';
    CALL Cargar_Log(45000,'El ID 1 de UsuarioAcceso no pertenece al usuario
admin','[SQL] Inicializador()',NULL);
  END IF;
END //
```

```
-- CREAR CLIENTE
CREATE PROCEDURE `Crear_Cliente`(
    IN `Nombre_IN` VARCHAR(40),
    IN `Tipo_IN` CHAR(1),
    IN `Rubro_IN` VARCHAR(50),
    IN `Director_IN` VARCHAR(40),
    IN `ID_Usuario_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
    DECLARE id_cliente INT DEFAULT 0;
    DECLARE validador TINYINT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
     GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
     ROLLBACK;
     BEGIN
        DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
       CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Crear_Cliente()',ID_Usuario_IN);
     END;
     IF validador THEN
        SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
     END IF;
     RESIGNAL;
   END;

   IF ((Tipo_IN IS NULL) OR (UPPER(Tipo_IN) NOT IN ('P','E'))) THEN
      SELECT 'El tipo de cliente debe ser P (Persona) o E (Empresa).' AS 'ADVERTENCIA';
     CALL Cargar_Log(45000,'Se intentó crear un cliente con un tipo inválido','[SQL]
Crear_Cliente()',ID_Usuario_IN);
   ELSEIF ((UPPER(Tipo_IN) = 'E') AND ((Rubro_IN IS NULL) OR (Director_IN IS NULL)))
THEN
      SELECT 'Una empresa necesita rubro y director.' AS 'ADVERTENCIA';
     CALL Cargar_Log(45000,'Se intentó crear una empresa sin rubro o sin director','[SQL]
Crear_Cliente()',ID_Usuario_IN);
   ELSE
      START TRANSACTION;
        INSERT INTO UsuarioCliente (nombre) VALUES (Nombre_IN);
        SET id_cliente = LAST_INSERT_ID();
```

```
        IF UPPER(Tipo_IN) = 'E' THEN
          INSERT INTO Empresa (rubro, director, idCliente) VALUES (Rubro_IN, Director_IN,
id_cliente);
        ELSE
          INSERT INTO Persona (idCliente) VALUES (id_cliente);
        END IF;
        CALL Cargar_Log(0,CONCAT('Se creó el cliente ',Nombre_IN,' (tipo
',UPPER(Tipo_IN),') con ID = ',id_cliente),'[SQL] Crear_Cliente()',ID_Usuario_IN);
     COMMIT;
  END IF;
END //


-- MODIFICAR CLIENTE
CREATE PROCEDURE `Modificar_Cliente`(
   IN `ID_Cliente_IN` INT,
   IN `Nuevo_Nombre_IN` VARCHAR(40),
   IN `Nuevo_Rubro_IN` VARCHAR(50),
   IN `Nuevo_Director_IN` VARCHAR(40),
   IN `ID_Usuario_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
   DECLARE validador TINYINT DEFAULT 0;

   DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
     GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
     ROLLBACK;
     BEGIN
        DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
       CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Modificar_Cliente()',ID_Usuario_IN);
     END;
     IF validador THEN
        SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
     END IF;
     RESIGNAL;
  END;

  IF EXISTS (SELECT 1 FROM UsuarioCliente WHERE idCliente = ID_Cliente_IN) THEN
     START TRANSACTION;
```

```
        UPDATE UsuarioCliente SET nombre = Nuevo_Nombre_IN WHERE idCliente =
ID_Cliente_IN;
        UPDATE Empresa SET rubro = COALESCE(Nuevo_Rubro_IN,rubro), director =
COALESCE(Nuevo_Director_IN,director) WHERE idCliente = ID_Cliente_IN;
        CALL Cargar_Log(0,CONCAT('Se modificó correctamente al cliente ID =
',ID_Cliente_IN),'[SQL] Modificar_Cliente()',ID_Usuario_IN);
     COMMIT;
  ELSE
      SELECT 'Cliente indicado no existente en el sistema' AS 'ADVERTENCIA';
     CALL Cargar_Log(45000,CONCAT('Se intentó modificar un cliente inexistente, ID =
',ID_Cliente_IN),'[SQL] Modificar_Cliente()',ID_Usuario_IN);
  END IF;
END //


-- ELIMINAR CLIENTE
CREATE PROCEDURE `Eliminar_Cliente`(
   IN `ID_Cliente_IN` INT,
   IN `ID_Usuario_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
   DECLARE validador TINYINT DEFAULT 0;

   DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
     GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
     ROLLBACK;
     BEGIN
        DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
       CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Eliminar_Cliente()',ID_Usuario_IN);
     END;
     IF validador THEN
        SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
     END IF;
     RESIGNAL;
  END;

  IF EXISTS (SELECT 1 FROM UsuarioCliente WHERE idCliente = ID_Cliente_IN) THEN
     START TRANSACTION;
        DELETE FROM Empresa WHERE idCliente = ID_Cliente_IN;
       DELETE FROM Persona WHERE idCliente = ID_Cliente_IN;
```

```
        DELETE FROM Solicitud WHERE idCliente = ID_Cliente_IN;
        DELETE FROM estado WHERE idCliente = ID_Cliente_IN;
        DELETE FROM UsuarioCliente WHERE idCliente = ID_Cliente_IN;
        CALL Cargar_Log(0,CONCAT('Se eliminó exitosamente al cliente ID =
',ID_Cliente_IN,' del sistema'),'[SQL] Eliminar_Cliente()',ID_Usuario_IN);
      COMMIT;
    ELSE
       SELECT 'Cliente indicado no existente en el sistema' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000,CONCAT('Se intentó eliminar un cliente inexistente, ID =
',ID_Cliente_IN),'[SQL] Eliminar_Cliente()',ID_Usuario_IN);
    END IF;
END //


-- BUSCAR USUARIOS
CREATE PROCEDURE `Buscar_Usuarios`(
    IN `Nombre_IN` VARCHAR(40),
    IN `Clave_IN` VARCHAR(200),
    IN `ID_Usuario_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
    DECLARE validador TINYINT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
      BEGIN
         DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
        CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Buscar_Usuarios()',ID_Usuario_IN);
      END;
      IF validador THEN
         SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
      END IF;
      RESIGNAL;
    END;

    IF (Nombre_IN IS NULL) THEN
       SELECT idUsuarioAcceso, nombreUsuario, clave, salt, numeroIntentos, ultimoIntento
FROM UsuarioAcceso;
    ELSEIF (Clave_IN IS NULL) THEN
```

```
      SELECT idUsuarioAcceso, nombreUsuario, clave, salt, numeroIntentos, ultimoIntento
FROM UsuarioAcceso WHERE (nombreUsuario = Nombre_IN);
  ELSE
      SELECT idUsuarioAcceso, nombreUsuario, clave, salt, numeroIntentos, ultimoIntento
FROM UsuarioAcceso WHERE ((nombreUsuario = Nombre_IN) AND (clave = Clave_IN));
  END IF;
END //


-- EXTRACTOR USUARIOS
CREATE PROCEDURE `Extractor_Usuarios`(
    IN `ID_Usuario_Buscado_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
    DECLARE validador TINYINT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
     GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
     BEGIN
        DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
       CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL]
Extractor_Usuarios()',ID_Usuario_Buscado_IN);
     END;
     IF validador THEN
        SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
     END IF;
     RESIGNAL;
  END;

  SELECT idUsuarioAcceso, nombreUsuario, clave, salt, numeroIntentos, ultimoIntento
FROM UsuarioAcceso WHERE idUsuarioAcceso = ID_Usuario_Buscado_IN;
END //


-- REINICIADOR DE INTENTOS
CREATE PROCEDURE `Reiniciador_Intentos`(
    IN `ID_Usuario_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
```

```
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
    DECLARE ult_fecha DATE;
    DECLARE validador TINYINT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
      ROLLBACK;
      BEGIN
         DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
        CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Reiniciador_Intentos()',ID_Usuario_IN);
      END;
      IF validador THEN
         SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
      END IF;
      RESIGNAL;
   END;

   IF EXISTS (SELECT 1 FROM UsuarioAcceso WHERE idUsuarioAcceso = ID_Usuario_IN)
THEN
       SELECT ultimoIntento INTO ult_fecha FROM UsuarioAcceso WHERE idUsuarioAcceso
= ID_Usuario_IN;
      IF ((ult_fecha IS NOT NULL) AND (DATEDIFF(CURRENT_DATE(),ult_fecha) > 7))
THEN
         START TRANSACTION;
           UPDATE UsuarioAcceso SET numeroIntentos = 0 WHERE idUsuarioAcceso =
ID_Usuario_IN;
            CALL Cargar_Log(0,CONCAT('Se reinició el contador de intentos del usuario ID =
',ID_Usuario_IN),'[SQL] Reiniciador_Intentos()',ID_Usuario_IN);
         COMMIT;
      END IF;
   ELSE
       SELECT 'Usuario indicado no existente en el sistema' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000,CONCAT('Se intentó reiniciar los intentos de un usuario
inexistente, ID = ',ID_Usuario_IN),'[SQL] Reiniciador_Intentos()',NULL);
   END IF;
END //


-- CREAR USUARIO
CREATE PROCEDURE `Crear_Usuario`(
    IN `Nombre_IN` VARCHAR(40),
    IN `Clave_IN` VARCHAR(200),
```

```
    IN `Salt_IN` VARCHAR(100),
    IN `ID_Usuario_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
    DECLARE id_nuevo INT DEFAULT 0;
    DECLARE validador TINYINT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
      ROLLBACK;
      BEGIN
         DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
        CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Crear_Usuario()',ID_Usuario_IN);
      END;
      IF validador THEN
         SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
      END IF;
      RESIGNAL;
   END;

   IF ((ID_Usuario_IN IS NULL) OR (ID_Usuario_IN != 1)) THEN
       SELECT 'Solo el administrador puede crear usuarios.' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000,'Un usuario que no es administrador intentó crear un
usuario','[SQL] Crear_Usuario()',ID_Usuario_IN);
   ELSEIF ((Nombre_IN IS NULL) OR (TRIM(Nombre_IN) = '') OR (Clave_IN IS NULL) OR
(Clave_IN = '') OR (Salt_IN IS NULL) OR (Salt_IN = '')) THEN
       SELECT 'El usuario necesita nombre, clave y salt.' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000,'Se intentó crear un usuario sin nombre, clave o salt','[SQL]
Crear_Usuario()',ID_Usuario_IN);
   ELSEIF EXISTS (SELECT 1 FROM UsuarioAcceso WHERE nombreUsuario =
Nombre_IN) THEN
       SELECT 'Ya existe un usuario con ese nombre.' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000,CONCAT('Se intentó crear un usuario con un nombre ya
existente: ',Nombre_IN),'[SQL] Crear_Usuario()',ID_Usuario_IN);
   ELSE
       START TRANSACTION;
          INSERT INTO UsuarioAcceso (nombreUsuario, clave, salt, numeroIntentos,
ultimoIntento)
         VALUES (Nombre_IN, Clave_IN, Salt_IN, 0, CURRENT_DATE());
         SET id_nuevo = LAST_INSERT_ID();
```

```
        CALL Cargar_Log(0,CONCAT('Se creó el usuario ',Nombre_IN,' con ID =
',id_nuevo),'[SQL] Crear_Usuario()',ID_Usuario_IN);
      COMMIT;
    END IF;
END //


-- SOLICITAR CREDITO
CREATE PROCEDURE `Solicitar_Credito`(
    IN `ID_Cliente_IN` INT,
    IN `Nombre_IN` VARCHAR(40),
    IN `Ingreso_IN` INT,
    IN `Gastos_IN` DECIMAL(10,2)
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
    DECLARE id_solicitud INT DEFAULT 0;
    DECLARE validador TINYINT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
      ROLLBACK;
      BEGIN
         DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
        CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Solicitar_Credito()',NULL);
      END;
      IF validador THEN
         SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
      END IF;
      RESIGNAL;
    END;

   IF ((ID_Cliente_IN IS NULL) OR (Nombre_IN IS NULL) OR (Ingreso_IN IS NULL) OR
(Gastos_IN IS NULL)) THEN
       SELECT 'La solicitud necesita cliente, nombre, ingresos y gastos.' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000,'Se intentó solicitar un crédito con datos incompletos','[SQL]
Solicitar_Credito()',NULL);
   ELSEIF ((Ingreso_IN < 0) OR (Gastos_IN < 0)) THEN
       SELECT 'Los ingresos y los gastos no pueden ser negativos.' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000,CONCAT('Se intentó solicitar un crédito con ingresos o gastos
negativos, cliente ID = ',ID_Cliente_IN),'[SQL] Solicitar_Credito()',NULL);
```

```
  ELSEIF NOT EXISTS (SELECT 1 FROM UsuarioCliente WHERE idCliente =
ID_Cliente_IN) THEN
      SELECT 'No existe un cliente con dicha ID.' AS 'ADVERTENCIA';
     CALL Cargar_Log(45000,CONCAT('Se intentó solicitar un crédito para un cliente
inexistente, ID = ',ID_Cliente_IN),'[SQL] Solicitar_Credito()',NULL);
  ELSEIF EXISTS (SELECT 1 FROM Solicitud WHERE idCliente = ID_Cliente_IN AND
estadoSolicitud = 'Pendiente') THEN
      SELECT 'El cliente ya tiene una solicitud pendiente.' AS 'ADVERTENCIA';
     CALL Cargar_Log(45000,CONCAT('Se intentó solicitar un crédito para un cliente con
una solicitud pendiente, ID = ',ID_Cliente_IN),'[SQL] Solicitar_Credito()',NULL);
  ELSE
      START TRANSACTION;
        UPDATE UsuarioCliente SET nombre = Nombre_IN WHERE idCliente =
ID_Cliente_IN;
       INSERT INTO Solicitud (ingresoAcutla, gastoAnual, estadoSolicitud, fechaSolicitud,
idCliente)
       VALUES (Ingreso_IN, Gastos_IN, 'Pendiente', CURRENT_DATE(), ID_Cliente_IN);
       SET id_solicitud = LAST_INSERT_ID();
       CALL Cargar_Log(0,CONCAT('Se registró la solicitud de crédito ID = ',id_solicitud,' del
cliente ID = ',ID_Cliente_IN),'[SQL] Solicitar_Credito()',NULL);
     COMMIT;
  END IF;
END //


-- ASIGNADOR ESTADO (Solicitud)
CREATE PROCEDURE `Asignador_Estado`(
   IN `ID_Solicitud_IN` INT,
   IN `Opcion_IN` VARCHAR(10),
   IN `ID_Usuario_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
   DECLARE estado_actual VARCHAR(20) DEFAULT NULL;
   DECLARE validador TINYINT DEFAULT 0;

   DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
     GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
     ROLLBACK;
     BEGIN
        DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
       CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Asignador_Estado()',ID_Usuario_IN);
```

```
    END;
    IF validador THEN
      SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
    END IF;
    RESIGNAL;
  END;

  IF ((Opcion_IN IS NULL) OR (Opcion_IN NOT IN ('Aprobado','Rechazado'))) THEN
     SELECT 'La opción debe ser Aprobado o Rechazado.' AS 'ADVERTENCIA';
    CALL Cargar_Log(45000,'Se intentó asignar una opción inválida a una solicitud','[SQL]
Asignador_Estado()',ID_Usuario_IN);
  ELSEIF NOT EXISTS (SELECT 1 FROM Solicitud WHERE idSolicitud = ID_Solicitud_IN)
THEN
     SELECT 'No existe una solicitud con dicha ID.' AS 'ADVERTENCIA';
    CALL Cargar_Log(45000,'Se intentó gestionar una solicitud inexistente','[SQL]
Asignador_Estado()',ID_Usuario_IN);
  ELSE
     SELECT estadoSolicitud INTO estado_actual FROM Solicitud WHERE idSolicitud =
ID_Solicitud_IN;
    IF estado_actual != 'Pendiente' THEN
      SELECT 'No se puede cambiar el estado de una solicitud ya gestionada.' AS
'ADVERTENCIA';
      CALL Cargar_Log(45000,CONCAT('Se intentó cambiar el estado de una solicitud ya
gestionada, ID = ',ID_Solicitud_IN),'[SQL] Asignador_Estado()',ID_Usuario_IN);
    ELSE
      START TRANSACTION;
        UPDATE Solicitud SET estadoSolicitud = Opcion_IN WHERE idSolicitud =
ID_Solicitud_IN;
        CALL Cargar_Log(0,CONCAT('Se cambió el estado de la solicitud ID =
',ID_Solicitud_IN,' a ',Opcion_IN),'[SQL] Asignador_Estado()',ID_Usuario_IN);
      COMMIT;
    END IF;
  END IF;
END //


-- EXTRACTOR SOLICITUD
CREATE PROCEDURE `Extractor_Solicitud`(
   IN `ID_Solicitud_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
   DECLARE validador TINYINT DEFAULT 0;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
    BEGIN
      DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
      CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Extractor_Solicitud()',NULL);
    END;
    IF validador THEN
      SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
    END IF;
    RESIGNAL;
  END;

  SELECT s.idSolicitud, s.idCliente, uc.nombre, s.ingresoAcutla, s.gastoAnual, uc.historial,
s.estadoSolicitud, s.fechaSolicitud
  FROM Solicitud s
  INNER JOIN UsuarioCliente uc ON uc.idCliente = s.idCliente
  WHERE s.idSolicitud = ID_Solicitud_IN;
END //


-- BUSCAR SOLICITUD
CREATE PROCEDURE `Buscar_Solicitud`(
    IN `Bandera_IN` BOOLEAN,
    IN `Estado_Solicitud_IN` VARCHAR(20),
    IN `Fecha_Solicitud_IN` DATE,
    IN `Men_Mas_Fec_IN` CHAR(1),
    OUT `Mensaje_OUT` VARCHAR(200),
    OUT `Exito_OUT` BOOLEAN
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
    DECLARE operador CHAR(1) DEFAULT '=';
    DECLARE cantidad INT DEFAULT 0;
    DECLARE validador TINYINT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
    BEGIN
      DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
```

```
        CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Buscar_Solicitud()',NULL);
      END;
      IF validador THEN
         SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
      END IF;
      RESIGNAL;
   END;

   SET Exito_OUT = 0;
   SET Mensaje_OUT = '';

   IF (Bandera_IN IS NULL) THEN
      SET Mensaje_OUT = 'La bandera debe ser true o false.';
   ELSEIF ((Estado_Solicitud_IN IS NOT NULL) AND (Estado_Solicitud_IN NOT IN
('Pendiente','Aprobado','Rechazado'))) THEN
      SET Mensaje_OUT = 'El estado debe ser Pendiente, Aprobado o Rechazado.';
   ELSEIF ((Bandera_IN) AND ((Estado_Solicitud_IN IS NULL) OR (Fecha_Solicitud_IN IS
NULL))) THEN
      SET Mensaje_OUT = 'Para gestionar una solicitud hacen falta el estado y la fecha.';
   ELSEIF ((NOT Bandera_IN) AND (Men_Mas_Fec_IN IS NOT NULL) AND
(Men_Mas_Fec_IN NOT IN ('<','=','>'))) THEN
      SET Mensaje_OUT = 'El criterio de fecha debe ser <, = o >.';
   ELSE
      IF Bandera_IN THEN
         SET operador = '=';
      ELSE
         SET operador = COALESCE(Men_Mas_Fec_IN,'=');
      END IF;

      SELECT COUNT(*) INTO cantidad
      FROM Solicitud s
      WHERE ((Estado_Solicitud_IN IS NULL) OR (s.estadoSolicitud = Estado_Solicitud_IN))
      AND ((Fecha_Solicitud_IN IS NULL)
         OR ((operador = '=') AND (s.fechaSolicitud = Fecha_Solicitud_IN))
        OR ((operador = '<') AND (s.fechaSolicitud < Fecha_Solicitud_IN))
        OR ((operador = '>') AND (s.fechaSolicitud > Fecha_Solicitud_IN)));

      IF cantidad = 0 THEN
         SET Exito_OUT = NOT Bandera_IN;
        SET Mensaje_OUT = 'No se encontraron solicitudes con esos datos.';
      ELSE
         SET Exito_OUT = 1;
        SET Mensaje_OUT = 'Búsqueda realizada correctamente.';

        IF Bandera_IN THEN
```

```
          SELECT s.idSolicitud, s.idCliente, uc.nombre, s.ingresoAcutla, s.gastoAnual,
s.estadoSolicitud, s.fechaSolicitud
          FROM Solicitud s
          INNER JOIN UsuarioCliente uc ON uc.idCliente = s.idCliente
          WHERE ((Estado_Solicitud_IN IS NULL) OR (s.estadoSolicitud =
Estado_Solicitud_IN))
          AND ((Fecha_Solicitud_IN IS NULL)
             OR ((operador = '=') AND (s.fechaSolicitud = Fecha_Solicitud_IN))
            OR ((operador = '<') AND (s.fechaSolicitud < Fecha_Solicitud_IN))
            OR ((operador = '>') AND (s.fechaSolicitud > Fecha_Solicitud_IN)))
          ORDER BY s.idSolicitud ASC
          LIMIT 1;
        ELSE
          SELECT s.idSolicitud, s.idCliente, uc.nombre, s.ingresoAcutla, s.gastoAnual,
s.estadoSolicitud, s.fechaSolicitud
          FROM Solicitud s
          INNER JOIN UsuarioCliente uc ON uc.idCliente = s.idCliente
          WHERE ((Estado_Solicitud_IN IS NULL) OR (s.estadoSolicitud =
Estado_Solicitud_IN))
          AND ((Fecha_Solicitud_IN IS NULL)
             OR ((operador = '=') AND (s.fechaSolicitud = Fecha_Solicitud_IN))
            OR ((operador = '<') AND (s.fechaSolicitud < Fecha_Solicitud_IN))
            OR ((operador = '>') AND (s.fechaSolicitud > Fecha_Solicitud_IN)))
          ORDER BY s.idSolicitud ASC;
        END IF;
      END IF;
   END IF;
END //


-- EXTRAER PERSONA
CREATE PROCEDURE `Extraer_Persona`(
    IN `ID_Buscado_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
    DECLARE validador TINYINT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
      BEGIN
         DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
```

```
        CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Extraer_Persona()',NULL);
      END;
      IF validador THEN
         SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
      END IF;
      RESIGNAL;
   END;

   SELECT uc.nombre, s.ingresoAcutla AS ingresos, s.gastoAnual AS gastos, uc.historial,
s.fechaSolicitud AS fechaHistorial
   FROM Persona p
   INNER JOIN UsuarioCliente uc ON uc.idCliente = p.idCliente
   LEFT JOIN Solicitud s ON s.idCliente = uc.idCliente
   WHERE p.idPersona = ID_Buscado_IN
   ORDER BY s.fechaSolicitud DESC, s.idSolicitud DESC;
END //


-- EXTRAER EMPRESA
CREATE PROCEDURE `Extraer_Empresa`(
    IN `ID_Buscado_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
    DECLARE validador TINYINT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
      BEGIN
         DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
        CALL Cargar_Log(@errno,CONCAT('ERROR:
[',@sqlstate,':',@errno,']\n',LEFT(@text,200)),'[SQL] Extraer_Empresa()',NULL);
      END;
      IF validador THEN
         SELECT CONCAT('ERROR: [',@sqlstate,':',@errno,']\n',@text,'\n(Además falló el
registro en infoLog)') AS 'ERROR';
      END IF;
      RESIGNAL;
   END;
```

```
  SELECT uc.nombre, s.ingresoAcutla AS ingresos, s.gastoAnual AS gastos, uc.historial,
s.fechaSolicitud AS fechaHistorial, e.rubro, e.director
  FROM Empresa e
  INNER JOIN UsuarioCliente uc ON uc.idCliente = e.idCliente
  LEFT JOIN Solicitud s ON s.idCliente = uc.idCliente
  WHERE e.idEmpresa = ID_Buscado_IN
  ORDER BY s.fechaSolicitud DESC, s.idSolicitud DESC;
END //


-- REGISTRAR INTENTO FALLIDO (login)
CREATE PROCEDURE `Registrar_Intento_Fallido`(
   IN `ID_Usuario_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
   DECLARE validador TINYINT DEFAULT 0;

   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
      ROLLBACK;
      BEGIN
         DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
         CALL Cargar_Log(@errno, CONCAT('ERROR: [', @sqlstate, ':', @errno, ']\n',
LEFT(@text, 200)), '[SQL] Registrar_Intento_Fallido()', ID_Usuario_IN);
      END;
      IF validador THEN
         SELECT CONCAT('ERROR: [', @sqlstate, ':', @errno, ']\n', @text, '\n(Además falló el
registro en infoLog)') AS 'ERROR';
      END IF;
      RESIGNAL;
   END;

   IF EXISTS (SELECT 1 FROM UsuarioAcceso WHERE idUsuarioAcceso =
ID_Usuario_IN) THEN
      START TRANSACTION;
      UPDATE UsuarioAcceso
      SET numeroIntentos = numeroIntentos + 1, ultimoIntento = CURRENT_DATE()
      WHERE idUsuarioAcceso = ID_Usuario_IN;
      COMMIT;
   ELSE
      SELECT 'Usuario indicado no existente en el sistema' AS 'ADVERTENCIA';
   END IF;
```

```
END //


-- MODIFICAR USUARIO
CREATE PROCEDURE `Modificar_Usuario`(
   IN `ID_Usuario_IN` INT,
   IN `Nuevo_Nombre_IN` VARCHAR(40),
   IN `Nueva_Clave_IN` VARCHAR(200),
   IN `Nuevo_Salt_IN` VARCHAR(100),
   IN `ID_Actor_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
   DECLARE validador TINYINT DEFAULT 0;

   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
      ROLLBACK;
      BEGIN
         DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
         CALL Cargar_Log(@errno, CONCAT('ERROR: [', @sqlstate, ':', @errno, ']\n',
LEFT(@text, 200)), '[SQL] Modificar_Usuario()', ID_Actor_IN);
      END;
      IF validador THEN
         SELECT CONCAT('ERROR: [', @sqlstate, ':', @errno, ']\n', @text, '\n(Además falló el
registro en infoLog)') AS 'ERROR';
      END IF;
      RESIGNAL;
   END;

   IF NOT EXISTS (SELECT 1 FROM UsuarioAcceso WHERE idUsuarioAcceso =
ID_Usuario_IN) THEN
      SELECT 'Usuario indicado no existente en el sistema' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000, CONCAT('Se intentó modificar un usuario inexistente, ID = ',
ID_Usuario_IN), '[SQL] Modificar_Usuario()', ID_Actor_IN);
   ELSEIF ((Nueva_Clave_IN IS NULL) <> (Nuevo_Salt_IN IS NULL)) THEN
      SELECT 'La clave y el salt deben cambiarse juntos.' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000, 'Se intentó modificar clave o salt por separado', '[SQL]
Modificar_Usuario()', ID_Actor_IN);
   ELSEIF ((ID_Usuario_IN = 1) AND (Nuevo_Nombre_IN IS NOT NULL) AND
(Nuevo_Nombre_IN <> 'admin')) THEN
      SELECT 'No se puede renombrar al administrador.' AS 'ADVERTENCIA';
```

```
      CALL Cargar_Log(45000, 'Se intentó renombrar al administrador', '[SQL]
Modificar_Usuario()', ID_Actor_IN);
   ELSEIF ((Nuevo_Nombre_IN IS NOT NULL) AND EXISTS (SELECT 1 FROM
UsuarioAcceso WHERE nombreUsuario = Nuevo_Nombre_IN AND idUsuarioAcceso <>
ID_Usuario_IN)) THEN
      SELECT 'Ya existe un usuario con ese nombre.' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000, CONCAT('Se intentó renombrar al usuario ID = ',
ID_Usuario_IN, ' con un nombre ya existente'), '[SQL] Modificar_Usuario()', ID_Actor_IN);
   ELSE
      START TRANSACTION;
      UPDATE UsuarioAcceso
      SET nombreUsuario = COALESCE(NULLIF(TRIM(Nuevo_Nombre_IN), ''),
nombreUsuario),
         clave = COALESCE(Nueva_Clave_IN, clave),
         salt = COALESCE(Nuevo_Salt_IN, salt)
      WHERE idUsuarioAcceso = ID_Usuario_IN;

      CALL Cargar_Log(0, CONCAT('Se modificó correctamente al usuario ID = ',
ID_Usuario_IN), '[SQL] Modificar_Usuario()', ID_Actor_IN);
      COMMIT;
   END IF;
END //


-- ELIMINAR USUARIO
CREATE PROCEDURE `Eliminar_Usuario`(
   IN `ID_Usuario_IN` INT,
   IN `ID_Actor_IN` INT
)
LANGUAGE SQL
NOT DETERMINISTIC
CONTAINS SQL
SQL SECURITY DEFINER
BEGIN
   DECLARE validador TINYINT DEFAULT 0;

   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      GET DIAGNOSTICS CONDITION 1 @sqlstate = RETURNED_SQLSTATE, @errno =
MYSQL_ERRNO, @text = MESSAGE_TEXT;
      ROLLBACK;
      BEGIN
         DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET validador = 1;
         CALL Cargar_Log(@errno, CONCAT('ERROR: [', @sqlstate, ':', @errno, ']\n',
LEFT(@text, 200)), '[SQL] Eliminar_Usuario()', ID_Actor_IN);
      END;
      IF validador THEN
```

```
        SELECT CONCAT('ERROR: [', @sqlstate, ':', @errno, ']\n', @text, '\n(Además falló el
registro en infoLog)') AS 'ERROR';
      END IF;
      RESIGNAL;
   END;

   IF (ID_Usuario_IN = 1) THEN
      SELECT 'No se puede eliminar al administrador.' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000, 'Se intentó eliminar al administrador', '[SQL]
Eliminar_Usuario()', ID_Actor_IN);
   ELSEIF NOT EXISTS (SELECT 1 FROM UsuarioAcceso WHERE idUsuarioAcceso =
ID_Usuario_IN) THEN
      SELECT 'Usuario indicado no existente en el sistema' AS 'ADVERTENCIA';
      CALL Cargar_Log(45000, CONCAT('Se intentó eliminar un usuario inexistente, ID = ',
ID_Usuario_IN), '[SQL] Eliminar_Usuario()', ID_Actor_IN);
   ELSE
      START TRANSACTION;
      UPDATE infoLog SET idUsuarioAcceso = NULL WHERE idUsuarioAcceso =
ID_Usuario_IN;
      DELETE FROM UsuarioAcceso WHERE idUsuarioAcceso = ID_Usuario_IN;

      CALL Cargar_Log(0, CONCAT('Se eliminó exitosamente al usuario ID = ',
ID_Usuario_IN), '[SQL] Eliminar_Usuario()', ID_Actor_IN);
      COMMIT;
   END IF;
END //

DELIMITER ;
```